

AES Implementation and Performance Evaluation

Mateo Oñate

School of Mathematical and Computational Sciences, Yachay Tech

email: mateo.onate@yachaytech.edu.ec

ABSTRACT We implemented AES-128, AES-192, and AES-256 in Python from the byte operations and key expansion defined in FIPS 197. The three versions work with 16-byte blocks and run 10, 12, and 14 rounds. Before measuring performance on our computer, we ran 30 automatic tests. All of them passed in 0.731 seconds, including finite-field arithmetic, S-box checks, inverse transformations, key expansion, published vectors, random round trips, invalid inputs, and benchmark calculations. We then measured 1, 10, and 100 MB inputs with three repetitions for every AES version. This produced 27 encryption/decryption pairs and 54 timed operations. On our Intel Core i3-1005G1 computer with Windows 10 and CPython 3.12.7, AES-128 encrypted 100 MB in 262.499 seconds on average, AES-192 in 310.263 seconds, and AES-256 in 354.333 seconds. Their corresponding encryption rates were 0.381, 0.322, and 0.282 MB/s. Decryption was slower in the three 100 MB tests. The measurements stayed close to proportional as the input size increased, while the versions with more rounds required more processing time.

INDEX TERMS AES, block cipher, finite-field arithmetic, key expansion, performance evaluation, Python.

I. Implementation

We organized the program into small modules so each part of AES can be checked separately. The cryptographic core depends only on the Python standard library. We store one AES state as a list of 16 byte values in column order. For row r and column c , the byte position is $4c + r$. This layout matches the state convention in FIPS 197 and keeps the round transformations close to the specification [1].

The three versions keep the same 128-bit block size. The key length changes the number of words in the initial key and the number of rounds. Table 1 summarizes the parameters implemented in our code.

Table 1: AES parameters implemented in the project.

Version	Key bytes	N_k	Rounds	Round keys
AES-128	16	4	10	11
AES-192	24	6	12	13
AES-256	32	8	14	15

A. Finite-field arithmetic and tables

AES performs byte arithmetic in $GF(2^8)$. Our `gf.py` module implements multiplication with shifts and XOR operations, reducing values with the AES polynomial

$$m(x) = x^8 + x^4 + x^3 + x + 1,$$

whose hexadecimal representation is `0x11B`. The `xtime` function multiplies one byte by x . The general multiplication routine repeats the same idea for eight bit positions.

We keep the standard AES S-box from FIPS 197 in `tables.py`. The program builds the inverse S-box by reversing the input-output mapping. It also calculates the product tables for 2, 3, 9, 11, 13, and 14 from our own finite-field multiplication routine. This choice keeps the transformations readable and avoids repeating the same byte multiplications inside every column operation.

B. Round transformations

We implemented the four AES transformations as separate functions. `SubBytes` replaces each state byte through the S-box. `ShiftRows` rotates rows by zero, one, two, and three positions. `MixColumns` combines the four bytes of each column through fixed field products. `AddRoundKey` applies XOR between the state and the current 16-byte round key. Since XOR is its own inverse, the same `AddRoundKey` function works during encryption and decryption.

The inverse path contains `InvSubBytes`, `InvShiftRows`, and `InvMixColumns`. For example, the direct column transformation works with coefficients 2, 3, 1, and 1, while the inverse transformation works with 14, 11, 13, and 9 in the corresponding matrix positions [1].

C. Key expansion

The key schedule accepts 16, 24, or 32 bytes. We calculate N_k from the key length and set the number of rounds to $N_k + 6$. Each new word normally comes from an earlier word and a transformed copy of the previous word. At positions divisible by N_k , we apply `RotWord`, `SubWord`, and the current round constant. AES-256 adds another `SubWord` step when the word index modulo eight equals four. The routine then groups the expanded words into 16-byte round keys.

The AES object expands the key once when we create the object. Encryption and decryption then reuse the same round keys for every block. This matters in the benchmark because key expansion stays outside the timed interval.

D. Encryption and decryption flow

Encryption starts with one `AddRoundKey`. Every middle round applies `SubBytes`, `ShiftRows`, `MixColumns`, and `AddRoundKey` in that order. The last round omits `MixColumns`, as required by AES. Decryption begins with the final round key, walks back-

ward through the expanded key schedule, and applies the inverse transformations in reverse order.

The block interface accepts exactly 16 bytes. The bulk interface accepts any byte string whose length is a multiple of 16 and processes the blocks independently. We did not add padding or an authenticated operating mode because this laboratory evaluates the AES core and its execution time, rather than a complete file-encryption format.

II. Validation

We ran the automatic test suite before starting the benchmark on our computer. All 30 tests passed in 0.731 seconds, with zero failures and zero errors. The tests cover both the smaller AES operations and the complete cipher. This gave us a quick way to check the implementation before spending more than an hour on the full set of measurements.

The finite-field test compares all 65,536 possible byte products against an independent polynomial multiplication routine. Another test derives all 256 S-box values from the mathematical definition and verifies the inverse table. The transformation tests check direct-inverse pairs, a published MixColumns example, state ordering, and the intermediate values from the first AES-128 round.

The key schedule tests compare expansion checkpoints for 128-, 192-, and 256-bit keys. They also verify the additional AES-256 substitution branch. At the cipher level, we compare our output with the three FIPS examples and twelve extra NIST block vectors. We then run 120 deterministic random encrypt-decrypt round trips across the three key lengths. Invalid key and block sizes must also raise the expected errors.

Table 2 lists the three standard examples that we also print in Exercise 1. The plaintext is 00112233445566778899AABBCCDDEEFF. Each key starts at byte 00 and increases by one until it reaches the required key length [3, 4].

Table 2: Published ciphertext checks for the three AES versions.

Variant	Ciphertext
AES-128	69C4E0D86A7B0430D8CDB78070B4C55A
AES-192	DDA97CA4864CDFE06EAF70A0EC0D7191
AES-256	8EA2B7CA516745BFEAFC49904B496089

We also ran an optional comparison against cryptography 46.0.0. That comparison processed 768 blocks in each direction across 48 different keys and reported a successful match. The external package does not participate in our AES core or the

timing experiment. It only provides an extra independent check of the final block results.

III. Experimental Setup

We collected the final measurements on our own computer. The recorded machine was an Intel Core i3-1005G1 at 1.20 GHz, with four logical processors, Windows 10 build 19045, and CPython 3.12.7. The benchmark itself ran sequentially in one process, so the results in this report describe the performance of the Python AES implementation on this machine without parallel execution.

We measured three input sizes: 1 MB, 10 MB, and 100 MB. In the benchmark, one MB equals 1,000,000 bytes. Each input length is a multiple of the 16-byte AES block. For every size, we measured AES-128, AES-192, and AES-256 three times. The complete grid contains 27 encryption/decryption pairs and 54 timed operations.

Table 3: Main benchmark settings from our computer.

Item	Recorded value
Runtime	CPython 3.12.7
CPU	Intel Core i3-1005G1 at 1.20 GHz
System	Windows 10, build 19045
Logical CPUs	4
Sizes	1, 10, 100 MB
Repetitions	3 per version and size
Timer	time.perf_counter
Process count	1
Seed	20260916
Warm-up	1 MB

The benchmark creates the input before starting the timer. For a given size, the three AES versions receive the same plaintext. The AES object and its expanded round keys are also prepared before timing. A 1 MB warm-up runs before the recorded measurements, and the order of AES-128, AES-192, and AES-256 changes between repetitions so one version is not always measured first.

The timer covers the encryption or decryption work, including output allocation and block processing. It does not include input generation, key expansion, file writes, hash calculations, or the final correctness comparison. Garbage collection stays enabled. After every pair, we compare the recovered bytes with the original plaintext and save SHA-256 values for the original, encrypted, and recovered data. All 27 recorded pairs returned the original input correctly.

IV. Results and Analysis

Table 4: Mean execution time, sample standard deviation, and throughput from three repetitions on our computer.

AES	MB	Enc. time, s	Dec. time, s	Enc. MB/s	Dec. MB/s
128	1	2.622 ± 0.134	2.668 ± 0.173	0.381	0.375
128	10	25.893 ± 0.768	27.385 ± 0.814	0.386	0.365
128	100	262.499 ± 26.165	276.291 ± 17.110	0.381	0.362
192	1	3.079 ± 0.100	3.220 ± 0.061	0.325	0.311
192	10	30.375 ± 0.465	33.106 ± 0.893	0.329	0.302
192	100	310.263 ± 20.536	336.786 ± 36.037	0.322	0.297
256	1	3.669 ± 0.360	3.778 ± 0.188	0.273	0.265
256	10	35.016 ± 0.271	37.635 ± 0.751	0.286	0.266
256	100	354.333 ± 13.136	381.068 ± 6.277	0.282	0.262

Table 4 shows the same general order in every input size. AES-128 finished first, AES-192 came next, and AES-256 took the most time. The difference is easiest to see with 100 MB because the runs are long enough for the round cost to stand out. AES-128 averaged 262.499 seconds for encryption. AES-192 averaged 310.263 seconds, which is 18.2 percent longer, and AES-256 averaged 354.333 seconds, which is 35.0 percent longer than AES-128.

The decryption results followed the same pattern. For 100 MB, the mean times were 276.291 seconds for AES-128, 336.786 seconds for AES-192, and 381.068 seconds for AES-256. Compared

with AES-128, this represents increases of 21.9 percent and 37.9 percent for AES-192 and AES-256.

These differences make sense when we compare the number of rounds. AES-128 runs 10 rounds, AES-192 runs 12, and AES-256 runs 14. Relative to AES-128, the round-count ratios are 1.20 and 1.40. In our 100 MB encryption measurements, the time ratios were 1.182 for AES-192 and 1.350 for AES-256. They are close to the round ratios, although they do not match exactly because the timer also includes Python-level block handling, function calls, and output allocation.

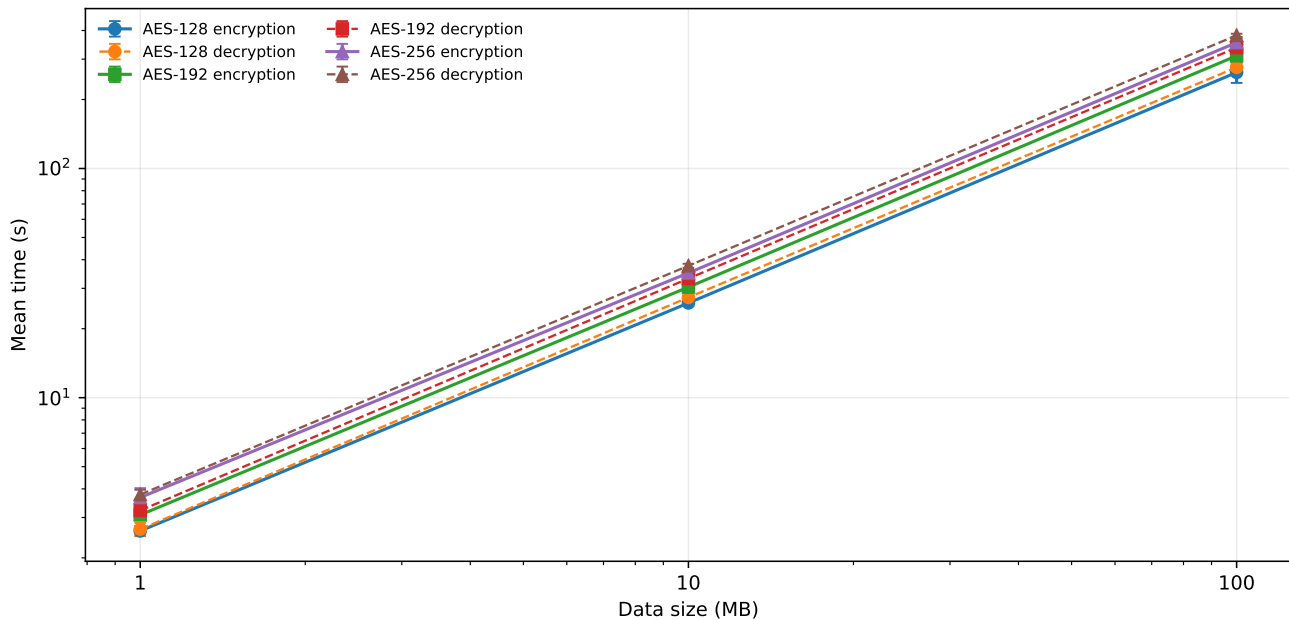


Figure 1: Mean encryption and decryption time measured on our computer. Error bars show the sample standard deviation from three repetitions. Both axes follow logarithmic scales.

Figure 1 shows that the running time grows almost in direct proportion to the amount of data. From 1 MB to 100 MB, mean encryption time increased by a factor of 100.11 for AES-128 and 100.76 for AES-192. AES-256 increased by a factor of 96.57. The small difference from an exact factor of 100 is expected in timed runs because the operating system and interpreter do not behave identically during every repetition.

The individual 100 MB measurements also show more variation than the 10 MB set. For example, the three AES-128 encryption runs were 248.388, 246.418, and 292.690 seconds. The third run was clearly slower and pushed the sample standard deviation to 26.165 seconds. AES-192 decryption showed a similar case. Its first two 100 MB runs were 315.812 and 316.148 seconds, while the third reached 378.397 seconds. We kept these values instead

of removing them because they are part of the measurements obtained on our computer.

Throughput stayed fairly steady as the amount of data increased. AES-128 encryption measured 0.381 MB/s at 1 MB, 0.386 MB/s

at 10 MB, and 0.381 MB/s at 100 MB. AES-192 stayed between 0.322 and 0.329 MB/s. AES-256 ranged from 0.273 to 0.286 MB/s. This is useful because it shows that the large increase in total time mainly comes from processing more blocks rather than from a large drop in the rate of the implementation.

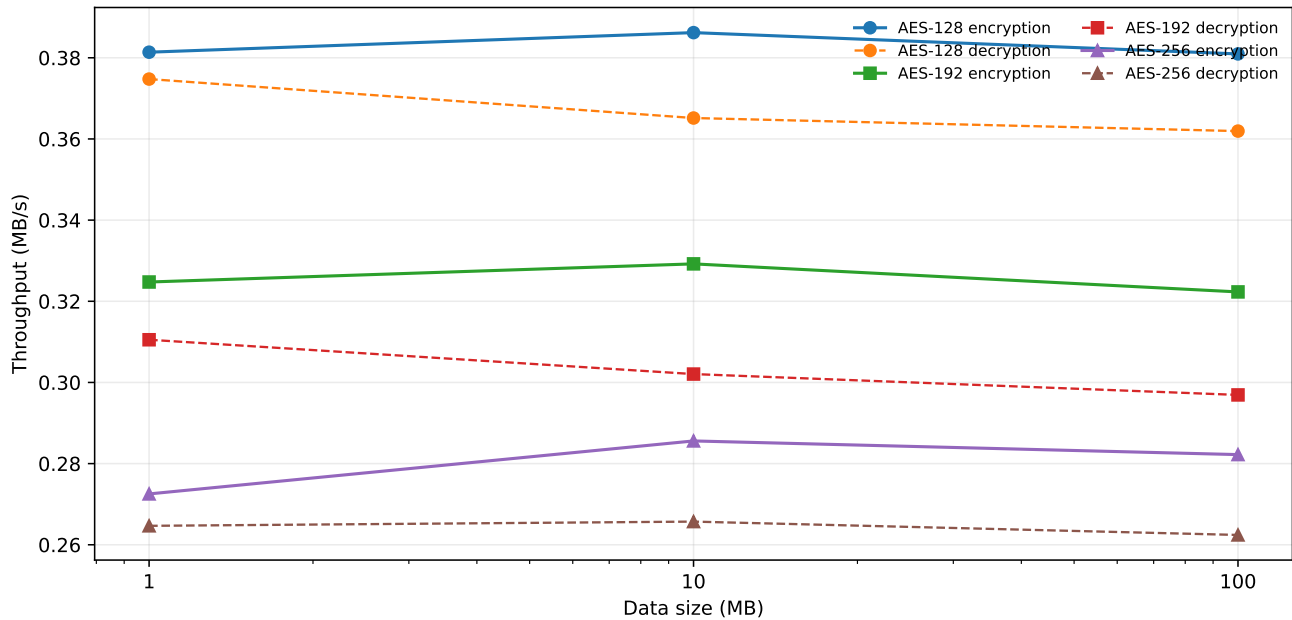


Figure 2: Encryption and decryption throughput calculated from the mean elapsed time for each input size.

At 100 MB, decryption took a little longer than encryption in all three versions. AES-128 decryption was about 5.3 percent slower than encryption, AES-192 was about 8.5 percent slower, and AES-256 was about 7.5 percent slower. Our inverse round contains `InvMixColumns`, which performs more field-product lookups than the direct `MixColumns` path. The benchmark measures the full decryption routine, so this difference also includes the rest of the inverse transformations and the Python overhead

around each block.

The 100 MB throughput numbers make the round cost visible in another way. Encryption reached 0.381 MB/s with AES-128, 0.322 MB/s with AES-192, and 0.282 MB/s with AES-256. Decryption reached 0.362, 0.297, and 0.262 MB/s. The key size itself does not change the 16-byte block size. What changes during block processing is the number of rounds.

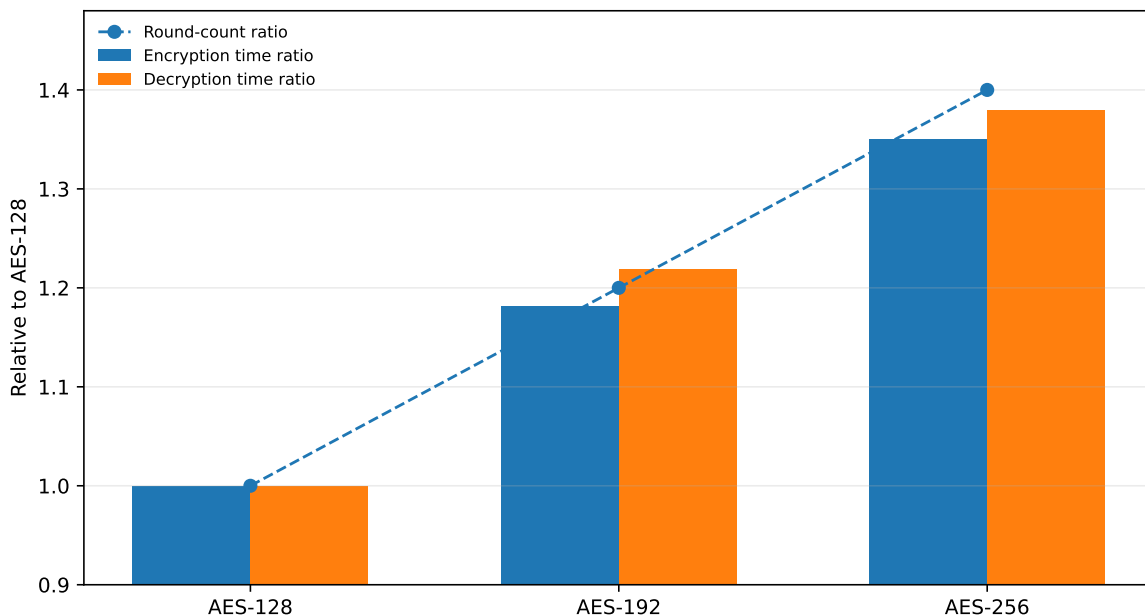


Figure 3: Measured 100 MB time relative to AES-128 compared with the ratio of AES round counts.

Figure 3 compares the measured 100 MB ratios with the number of rounds. The AES-192 encryption ratio of 1.182 stays close to its 1.20 round ratio. AES-256 reached 1.350 compared with a round ratio of 1.40. Decryption followed the same direction. The measurements are not meant to prove a fixed cost per round, but they show the effect clearly in our implementation.

Every saved benchmark row reports a successful round trip. The plaintext SHA-256 value and the recovered SHA-256 value match for all 27 pairs. This matters because a fast measurement would not be useful if the output failed to decrypt correctly. Together with the 30 passing tests and the published AES vectors, the benchmark results give us a consistent record of both correctness and performance on our own machine.

These numbers describe this Python implementation on the recorded computer. They are not performance figures for hardware AES instructions or optimized native libraries. The goal of the laboratory was to measure the implementation we wrote and to compare the three AES key sizes under the same conditions.

V. Conclusions

We completed AES-128, AES-192, and AES-256 with the same 16-byte block interface and separate code for finite-field arithmetic, round transformations, key expansion, encryption, and decryption. Before running the benchmark, our test suite completed 30 tests in 0.731 seconds with no failures. The published AES vectors matched the expected ciphertexts, and all 27 measured encryption/decryption pairs recovered the original input.

The measurements from our computer show a clear increase in execution time as the number of AES rounds grows. With 100 MB, AES-128 encryption averaged 262.499 seconds. AES-192 averaged 310.263 seconds and AES-256 averaged 354.333 seconds. The corresponding encryption rates were 0.381, 0.322, and

0.282 MB/s. Decryption followed the same order and remained slightly slower than encryption.

The throughput stayed reasonably stable between 1, 10, and 100 MB for each AES version. This matches what we expected from a block-by-block implementation, since a larger input mainly means processing more 16-byte blocks. Some 100 MB repetitions varied more than the shorter runs, especially AES-128 encryption and AES-192 decryption. We kept those measurements in the dataset because they reflect what happened during the experiment on our computer.

The JSON and CSV files keep the individual runs, machine information, benchmark settings, and correctness checks. This makes it possible to repeat the same experiment later and compare results without mixing measurements from another environment.

References

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, updated May 9, 2023.
- [2] Cryptography course, *Lab 3: AES Implementation and Evaluation*, laboratory handout, pp. 1–5, 2026.
- [3] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2001 edition, Appendix C.
- [4] National Institute of Standards and Technology, *AES Core Examples*, published worked block examples for AES-128, AES-192, and AES-256.
- [5] M. Oñate, *Laboratory 3 AES benchmark measurements*, experimental dataset, September 16, 2026. Files: `benchmark.json`, `mediciones.csv`, and `resumen.csv`.